

Ali0x01 - Combat Tank Agent

AI for Computer Games - Final Project Report

Group member	Contact
Gioia Zheng	zheng.2040439@studenti.uniroma1.it
ZeYang Fu	zeyangfu26@gmail.com
Gankhulug Bayaraa	gbayaraa@uef.fi

1. Introduction

The task was to train an agent for the two-player Atari Combat Tank game and submit it through the instructor-provided Agent interface. The tournament configuration contains a maze, visible tanks, no invisibility, and no billiard-style bullet rebounds. Each player receives +1 for hitting the opponent and -1 when hit. The main difficulty is that useful combat events are sparse: a policy can execute thousands of actions without receiving a non-zero reward.

Our objective was therefore broader than learning to shoot. The agent had to leave its starting corridor, navigate around fixed walls, find or reacquire a moving opponent, and continue acting after either tank was displaced by a hit. It also had to work from both player roles under the exact evaluation wrappers.

Environment used for all final evaluations

Component	Final setting
Game	PettingZoo Atari combat_tank_v2, parallel API
Rules	has_maze=True, is_invisible=False, billiard_hit=False
Observation	84 x 84 grayscale; max over 2 frames; 4-frame stack; 2 role indicators
Action space	Discrete(18)
Time handling	Frame skip 4; reward clipped to [-1, 1]
Official reward	+1 hit, -1 received hit, otherwise 0

The wrapper order and game settings above were kept identical in training, evaluation, visual playback, and packaging. Training-only reward shaping never changed tournament scoring.

2. Solution

2.1 Initial idea

The initial plan was imitation learning followed by reinforcement learning. A scripted teacher demonstrated movement, turning, and firing, after which PPO would improve the policy from game rewards. This gave a useful debugging baseline, but a fixed teacher route did not cover the complete maze and transferred its route bias to the learner.

2.2 PPO policy

The final learner used a convolutional PPO policy. Three convolution layers (32, 64, and 64 channels) encoded the six-channel official observation. A 512-unit projection produced logits for all 18 actions. PPO updates used the clipped objective $L = E[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$, with $\epsilon=0.1$ in the final branches. Training used CUDA and multiple parallel environments.

Because the two spawn positions and useful openings are asymmetric, one shared checkpoint was not consistently best for both roles. We therefore exported two role-specialized policies into one Agent. The two indicator planes appended by SuperSuit select the correct action head at inference time.

2.3 Navigation shaping

The maze was divided into coarse position cells. During exploration stages, entering a new cell added $\beta/\sqrt{N(c)}$ to the official reward, where $N(c)$ is the lifetime visit count for that role and cell. Long periods without physical displacement received a small penalty. The complete training reward was $r_{\text{train}} = r_{\text{official}} + r_{\text{visit}} - r_{\text{idle}}$. These terms encouraged exploration but were set to zero during model selection.

2.4 Opponent curriculum and inference

Training alternated between random opponents and frozen PPO snapshots. Later branches used mixed opponents and separate first-role and second-role fine-tuning. The learned PPO policies remained deliberately stochastic. Pure argmax inference collapsed some checkpoints to one repeated action, so the submitted Agent samples from the learned categorical distribution. This preserves the behavior used during training and uses all 18 legal actions.

Parameter	Final branch value
Optimizer / learning rate	Adam / 1e-5 for long fine-tuning
Rollout	4 or 8 environments; 512 or 1024 steps per environment
PPO epochs / batch	4 epochs / 256 samples
Discount / GAE	$\gamma=0.99$ / $\lambda=0.95$
Entropy coefficient	0.005-0.02 depending on branch
Overnight branches	1.5M exploit + 1.5M diverse + 0.75M per role

3. Development and Modifications

The project was developed by repeatedly watching full matches, measuring coverage and action use, and then changing one part of the curriculum. The table records the main observations and decisions.

Stage	Observed problem	Modification	Outcome
Random / basic PPO	Sparse reward and no reliable contact	Added scripted demonstrations and PPO initialization	Agent moved and fired, but copied a narrow opening route
Training vs random	Random opponent often stayed near spawn; learner overfit to one side and route	Added frozen-model and mixed opponents	More realistic encounters, but some stationary policies remained
Self-play	Both tanks sometimes rotated and fired without changing position	Added map-cell coverage and idle diagnostics	Made the failure measurable instead of relying only on score
Maze exploration	17/26 cells but about 84% idle; only 3 actions dominated	Tried frontier novelty and longer exploration	Coverage improved in some runs, combat did not; not selected
Frontier run	Final policy collapsed to 2 actions and 6/8 cells	Rejected final checkpoint and retained earlier candidates	Confirmed that later checkpoints are not automatically better
Waypoint teacher	Top routes reached 4 waypoints, bottom routes only 2; high idle fraction	Kept waypoint study as auxiliary pretraining only	Avoided submitting route-specific behavior
Long self-play	4.5M extra steps did not consistently improve held-out score	Selected checkpoints by opponent and role instead of training return	Older role specialists beat several later checkpoints
Tournament export	Argmax produced repeated actions	Sampled the PPO categorical distribution and exported two role heads	18/18 stochastic action coverage and stronger peer matches

Two design rules followed from these experiments. First, shaped return was used for learning but never for promotion. Second, every candidate was checked from both roles and against more than one opponent. This prevented us from selecting a model that looked active but only exploited one stationary opponent.

4. Numerical and Visual Results

4.1 Held-out official evaluation

The table below reports five games per setting using the official unshaped reward. The frozen opponent is the 250k self-play checkpoint. Because five games are a small sample and scores have high variance, these values were used as screening evidence rather than precise estimates.

Role / candidate	Random score	Frozen score	Cells vs random	Cells vs frozen
first_0 base	+0.20 +/- 0.40	+0.20 +/- 0.40	15.6	17.8
first_0 exploration (selected)	+0.40 +/- 1.02	+0.00 +/- 0.63	19.0	19.4
first_0 overnight diverse	-0.40 +/- 0.80	+0.20 +/- 0.40	19.0	18.0
first_0 role specialist	+0.20 +/- 0.98	+0.00 +/- 0.63	13.0	13.2
second_0 base	-0.40 +/- 1.36	+0.40 +/- 0.80	19.6	15.8
second_0 specialist (selected)	-0.80 +/- 1.17	+1.20 +/- 1.47	14.6	19.0
second_0 overnight exploit	-0.20 +/- 0.75	+0.60 +/- 1.36	11.8	14.2

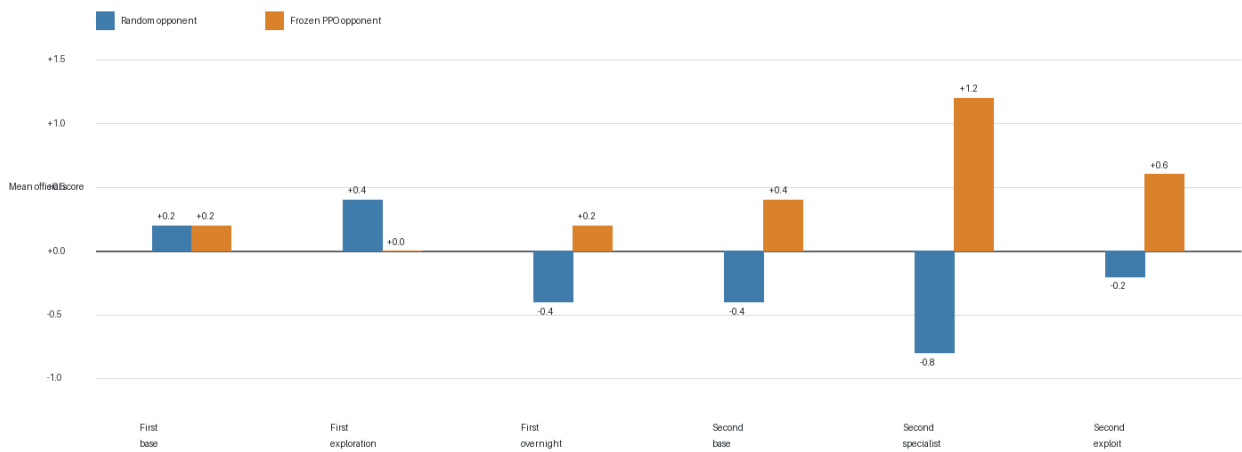


Figure 1. Mean official score for representative candidate branches. Blue bars use a random opponent; orange bars use the frozen PPO opponent. The selected role specialists were chosen for complementary behavior, not for one global average.

4.2 External peer matches

The selected first-role policy won all 3 peer-test games (+1 in each game). The selected second-role policy won 2 games and drew 1. These tests used the exact official observation pipeline and the peer's submitted Agent interface. They provided stronger opponent diversity than random-only testing, but the sample remains small.

Selected policy	Games	Wins	Draws	Losses
first_0 exploration checkpoint	3	3	0	0
second_0 role checkpoint	3	2	1	0

4.3 Visual behavior

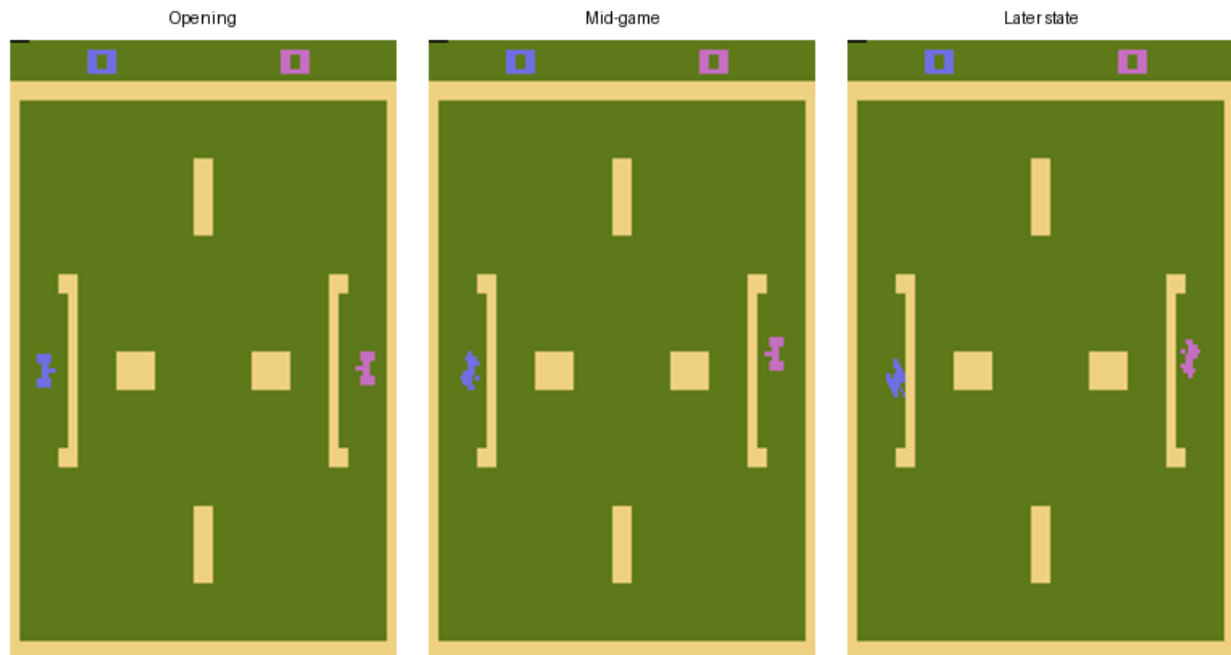


Figure 2. Frames from an official maze match. The agent must rotate inside the narrow spawn corridor, pass the vertical wall opening, and reacquire the opponent after movement or a hit displacement.

4.4 Remaining limitations

The score is still low in many random-opponent games, and several episodes end in draws. Navigation coverage does not guarantee useful line of sight, while combat fine-tuning can erase exploration. The feed-forward four-frame policy also has no explicit long-term map memory. Finally, stochastic sampling is necessary for the submitted policy; deterministic argmax diagnostics reveal severe action collapse in some checkpoints. These limitations are reported because they materially affect tournament reliability.

5. Conclusions

We reproduced the instructor's Combat Tank pipeline and trained a role-specialized PPO agent without changing the game rules or evaluation interface. The final solution combines a CNN policy, mixed-opponent training, exploration-only shaping, role-specific checkpoint selection, and stochastic tournament inference.

The most important result was methodological: more training steps did not reliably produce a stronger agent. Maze-only rewards could increase movement while reducing combat, and self-play could converge to repeated local behavior. Held-out official scoring, maze coverage, idle fraction, action coverage, visual playback, and peer matches were therefore all needed to choose a submission.

The selected pair performed well in the available peer test (five wins and one draw across six role-specific games), but its low and variable held-out scores show that the task is not solved. A future version should use recurrent memory or an explicit learned map, a larger opponent pool, and more held-out matches before checkpoint promotion.

6. Reproducibility

Artifact	Purpose
src/official_environment.py	Exact environment and training adapter
src/train_official_ppo.py	PPO training and checkpointing
src/evaluate_official_ppo.py	Unshaped role-specific evaluation
src/export_official_dual_submission.py	Two-role tournament export
scripts/overnight_official_13h.sh	Reproducible long-run curriculum

References

Course repository: <https://github.com/Hautamaki-lab/Summer-School-2026>
PettingZoo Combat Tank: https://pettingzoo.farama.org/environments/atari/combat_tank/
Schulman et al. (2017), Proximal Policy Optimization Algorithms.